CSF303: Competitive Programming

Software Engineering

Assignment 1 & 2

Submitted by

Student No: 02230300

Github link: https://github.com/liberationzany/SS2026_CSF303.git

Royal University of Bhutan, College of Science and Technology

Assignment 1

1. STL usage

```cpp
int main() {
    // ---------- Part (a): vector ----------
    int N;
    std::cout << "Enter number of integers: ";
    std::cin >> N;

    std::vector<int> vec(N);
    std::cout << "Enter " << N << " integers: ";
    for (int i = 0; i < N; ++i) {
        std::cin >> vec[i];
    }

    // Output in reverse using reverse iterators
    std::cout << "Vector in reverse: ";
    for (auto it = vec.rbegin(); it != vec.rend(); ++it) {
        std::cout << *it << " ";
    }
    std::cout << "\n";
```

In this implementation, it asks for N number of integers and store them in a vector. Instead of using a standard for loop I have implemented a vec.rbegin() and vec.rend(). rbegin() points to the last element in the array and we increment the iterator backwards toward the start of the vector as we are starting from the end.

```cpp
    std::deque<int> dq;
    int ops;
    std::cout << "Enter number of operations: ";
    std::cin >> ops;
    std::cout << "Operations (1: push_back, 2: push_front, 3: pop_front):\n";
    for (int i = 0; i < ops; ++i) {
        int type, val;
        std::cin >> type;
        if (type == 1) {
            std::cin >> val;
            dq.push_back(val);
        } else if (type == 2) {
            std::cin >> val;
            dq.push_front(val);
        } else if (type == 3) {
            if (!dq.empty()) {
                dq.pop_front();
            } else {
                std::cout << "Deque is empty, cannot pop.\n";
            }
        }
    }

    std::cout << "Deque contents: ";
    for (const auto &x : dq) {
        std::cout << x << " ";
    }
    std::cout << "\n";
```

This implementation provides a small interactive menu where a user can push_back elements to the end, push_front which adds element in the front or pop_front which removes the first element. This is an ideal container if one is building something like a undo/redo buffer or a job queue.

```
    constexpr int M = 5;  // fixed size
    std::array<int, M> arr;
    std::cout << "Enter " << M << " integers: ";
    for (int i = 0; i < M; ++i) {
        std::cin >> arr[i];
    }

    int sum = std::accumulate(arr.begin(), arr.end(), 0);
    std::cout << "Sum of array elements: " << sum << "\n";

    return 0;
}
```

This implementation iterates through the array and sums all the value starting from the index 0.

2. Bellman Ford

```cpp
#include <iostream>
#include <vector>
#include <limits>

using namespace std;

struct Edge {
    int u, v;
    long long w;
};

void bellmanFord(int V, int E, int src, const std::vector<Edge>& edges) {
    const long long INF = numeric_limits<long long>::max();
    vector<long long> dist(V, INF);
    dist[src] = 0;

    // Relax all edges V-1 times
    for (int i = 0; i < V - 1; ++i) {
        bool updated = false;
        for (const auto& e : edges) {
            if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) {
                dist[e.v] = dist[e.u] + e.w;
                updated = true;
            }
        }
        if (!updated) break; // Early exit if no relaxation
    }

    // Check for negative weight cycle
    bool hasNegCycle = false;
    for (const auto& e : edges) {
        if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) {
            hasNegCycle = true;
            break;
        }
    }

    if (hasNegCycle) {
        cout << "Graph contains a negative weight cycle reachable from source.\n";
    } else {
        cout << "Shortest distances from source " << src << ":\n";
        for (int i = 0; i < V; ++i) {
            cout << "Vertex " << i << " : ";
            if (dist[i] == INF)
                cout << "INF\n";
            else
                cout << dist[i] << "\n";
        }
    }
}

int main() {
    int V, E;
    cin >> V >> E;
    vector<Edge> edges(E);
    for (int i = 0; i < E; ++i) {
        cin >> edges[i].u >> edges[i].v >> edges[i].w;
    }
    int src;
    cin >> src;
    bellmanFord(V, E, src, edges);
    return 0;
}
```

3. Floyd-Warshall

```cpp
#include <iostream>
#include <vector>
#include <limits>

using namespace std;

const long long INF = numeric_limits<long long>::max();

void floydWarshall(vector<vector<long long>>& dist, int V) {
    // Initialize diagonal to 0 (already done if input ensures it)
    for (int k = 0; k < V; ++k) {
        for (int i = 0; i < V; ++i) {
            if (dist[i][k] == INF) continue;
            for (int j = 0; j < V; ++j) {
                if (dist[k][j] == INF) continue;
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    // Detect negative cycle: if any dist[i][i] < 0
    bool hasNegCycle = false;
    for (int i = 0; i < V; ++i) {
        if (dist[i][i] < 0) {
            hasNegCycle = true;
            break;
        }
    }

    if (hasNegCycle) {
        cout << "Graph contains a negative cycle.\n";
    } else {
        cout << "All-pairs shortest distance matrix:\n";
        for (int i = 0; i < V; ++i) {
            for (int j = 0; j < V; ++j) {
                if (dist[i][j] == INF)
                    cout << "INF ";
                else
                    cout << dist[i][j] << " ";
            }
            cout << "\n";
        }
    }
}

int main() {
    int V, E;
    cin >> V >> E;
    vector<vector<long long>> dist(V, vector<long long>(V, INF));
    for (int i = 0; i < V; ++i) dist[i][i] = 0;

    for (int i = 0; i < E; ++i) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        dist[u][v] = min(dist[u][v], w); // handle multiple edges
    }

    floydWarshall(dist, V);
    return 0;
}
```

This algorithm relies on dynamic programming and relaxation. Floyd-Warshall algorithm can handle negative weights because we consider all the intermediate vertices and the algorithm itself does not assume non-negative weights. However, the presence of a negative cycle means the shorted distance can be made small or undefined. That's why in presence of negative cycle, it fails.

Assignment 2

1. Bitmasking

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int N, K;
    cin >> N;
    vector<int> arr(N);
    for (int i = 0; i < N; ++i) cin >> arr[i];
    cin >> K;

    int count = 0;
    for (int mask = 0; mask < (1 << N); ++mask) {
        long long sum = 0;
        for (int i = 0; i < N; ++i) {
            if (mask & (1 << i)) {
                sum += arr[i];
            }
        }
        if (sum % K == 0) {
            count++;
        }
    }
    cout << "Number of subsets with sum divisible by " << K << ": " << count << "\n";
    return 0;
}
```

2. Johnson's Algorithm

Johnson's algorithm runs in $O(V^2 \log V + VE)$ when it uses Fibonacci heap and $O(VE \log V)$ with binary heap which compared to Floyd-warshall's $O(V^3)$ when E is much smaller than V^2. There, Johnson's algorithm is better for sparse graph.

Reweighting transforms edge weights to non-negative values so that Dijkstra's algorithm can be used. This is done by adding a dummy node connected to all vertices with zero weight, running Bellman-Ford to compute potential values $h(v)$ for each vertex, and then reweighting each edge (u, v, w) to $w' = w + h(u) - h(v) \geq 0$. Bellman-Ford detects any negative cycles before reweighting.

3. Arbitrage Detection in Currency Exchange

The vertices represent the currencies and the directed edge u → v with weight w represents the value of u when exchanging to v from u. A cycle $c_1 \to c_2 \to \cdots \to c_k \to c_1$ with product of rates > 1 represents an arbitrage opportunity.

First, we take the logarithm of each exchange rate which is -log (rate). If the product of exchange rate is greater than 1, and we take log on both the side and multiply it by -1 then the sum of negative logs becomes less than 0 indicating a negative weight cycle. Hence, this is how arbitrage detection reduces to negative cycle detection

Bellman Ford is used to detect negative cycles after the log transformation. It can find a negative cycle reachable from any source by adding a super-source connecting every vertex.

4. Edmond's Algorithm

Edmond's algorithm is used for finding a minimum weight directed spanning at r, where every vertex in the graph can be reached from r via directed edges and the total cost of the selected edges is as minimum as possible.

4. String Matching Algorithms

Index (i)	Pattern[0..i]	LPS[i]	Reason
0	A	0	No proper prefix = suffix
1	AB	0	No match
2	ABA	1	Prefix "A" = suffix "A"
3	ABAB	2	Prefix "AB" = suffix "AB"
4	ABABC	0	No match
5	ABABCA	1	"A" = "A"
6	ABABCAB	2	"AB" = "AB"
7	ABABCABA	3	"ABA" = "ABA"
8	ABABCABAB	2	"AB" = "AB" (longest proper)

When the hash of a pattern matches the hash of a text window, a character by character verification is performed to confirm the match because different strings can have the same hash.

Complexities:
Average case: O(N+M) where N is text length and M is pattern length
Worst case: O(NM) when many spurious hash collisions force verification each time.